
CATANBENCH: A REAL-TIME BENCHMARK FOR STRATEGIC REASONING IN SETTLERS OF CATAN

Vayun Gupta, Sharayu Janga, Dhruv Malpani, Rudy Juarez-Ovallos
University of Illinois Urbana Champaign

ABSTRACT

We present CatanBench, a benchmark for evaluating strategic decision making in the board game Settlers of Catan. Unlike benchmarks that primarily score agents by final game outcome, CatanBench evaluates individual decisions at the moment they are made. The benchmark covers short-term tactical moves, medium-term resource and build prioritization, and long-term strategic commitments such as pursuing Longest Road or Largest Army. Each decision is compared against the best legal alternative available in the current game state using heuristics. During live gameplay, agent actions automatically trigger task evaluators, persist task scores, and support both real-time telemetry and offline aggregate analysis.

1 INTRODUCTION

Many game benchmarks evaluate agents using final outcomes, but win/loss signals are too sparse to capture strategic reasoning. In Settlers of Catan, outcomes depend on hidden information, stochastic dice rolls, negotiation, and long-term dependencies, meaning strong decisions can still lose and weak ones may go unpunished. Existing benchmarks such as GameBench and Kaggle Game Arena rely on match outcomes or final outputs, and often assume simplified or perfect information, missing Catan’s mixed-information, multi-agent dynamics.

CatanBench addresses this gap by evaluating decisions during live gameplay. It defines 20 tasks spanning short-term (tactical moves like road placement or trades), medium-term (turn-level prioritization), and long-term (strategic commitments like opening placements or Longest Road) reasoning. Each action is scored against the best legal alternative at that moment, rather than a fixed answer.

Our contribution is a real-time 20 task benchmark that measures decision quality in context. Built on a full-stack Catan implementation with a Python agent API and deterministic rubrics, it records both per-decision and game-level metrics, enabling analysis of how agents play.

2 RELATED WORK

CatanBench is related to recent game-based benchmarks for evaluating strategic reasoning in language-model agents. GameBench evaluates the strategic reasoning abilities of LLM agents in game environments and is closest to CatanBench in its emphasis on strategic rather than purely static reasoning (Costarelli et al., 2024). CivBench evaluates AI agents in FreeCiv, a long-horizon adversarial environment designed to stress planning quality, adaptation under uncertainty and pressure, persistent strategy identity, execution efficiency, and stability across complete match trajectories rather than one-turn snapshots (Halevy, 2026). Kaggle Game Arena provides a broader framework around game environments, model harnesses, visualizers, and leaderboards, ranking models by outcome-oriented metrics such as Elo while using games as structured tests of strategic reasoning, long-term planning, and dynamic adaptation (Risdal, 2025). CatanBench differs from these benchmarks by scoring decision quality during a full game instead of relying primarily on match outcome, Elo, or trajectory-level summaries. The benchmark is not only a Catan environment; it is an instrumentation layer over gameplay that records whether an agent’s selected legal action is close to the best legal alternative under a task-specific rubric. This fills a complementary gap: CatanBench evaluates situated strategic judgment under changing board state, stochastic dice outcomes, hidden opponent resources, and negotiation, while preserving dense per-decision feedback across short, medium, and long-term Catan tasks.

3 BENCHMARK DESIGN

3.1 TASK DOMAIN AND SCOPE

CatanBench covers four categories of Catan decisions: (1) spatial placement (settlements and roads), (2) resource management (trading, discarding, bank trades, build vs. save), (3) opponent interaction (robber placement, victim selection, trade targeting, counteroffers, leader blocking), and (4) strategic commitments (e.g., pursuing Longest Road or Largest Army). The benchmark is designed to test decision quality rather than only game outcome. This includes individual legal decisions made during real gameplay, their task-specific scores, pass/fail thresholds, and aggregate performance over games. Natural language explanation grading is out of scope for this benchmark. No existing benchmark framework was used to design CatanBench.

3.2 TASK SUITE

Table 1 summarizes the 20 tasks by difficulty and reasoning horizon. It covers tasks over short, medium, and long-horizon reasoning during gameplay. Full task definitions are in the Appendix.

Initial settlement location selection. The input is a setup-phase snapshot including board layout, legal settlement vertices, adjacent resources and dice numbers, ports, occupied vertices, and legal road expansions. The output is a legal settlement vertex.

Each candidate is scored as:

$$\begin{aligned} \text{score} = & 0.40 \cdot \text{production} + 0.25 \cdot \text{diversity} \\ & + 0.20 \cdot \text{expansionAccess} + 0.15 \cdot \text{portSynergy}. \end{aligned} \quad (1)$$

Production uses dice probability weights; diversity measures resource variety; expansion access counts future road options; and port synergy captures trade efficiency. The selected vertex is compared to the best legal alternative with success threshold 0.75. Early placements determine long-term resource flow, expansion potential, and trade structure before any gameplay outcomes are observed.

Discard strategy after seven. The input is a player hand after a seven roll, discard requirement and estimated build priorities. The output is a legal discard set. All valid discard choices are evaluated by scoring the retained hand:

$$\text{retainedHandValue} = 0.55 \cdot \text{bestAffordable} + 0.30 \cdot \text{bestFuture} + 0.15 \cdot \text{diversity}. \quad (2)$$

The final score is the ratio of the chosen retained hand value to the best possible retained hand value. The pass threshold is 0.60. Agents must reason about opportunity cost under uncertainty, balancing immediate builds, future plans, and resource flexibility.

Delayed Longest Road pursuit. The input is a live state where a Longest Road contest is relevant (e.g., active competition, threshold road lengths, or proximity to a rival). The output is decision: pursue or defer, evaluated over a multi-turn horizon. Scoring compares both options using road growth, threat level, probability of securing Longest Road, and expansion efficiency. The pass threshold is 0.30 due to high stochasticity over intervening turns. This task is challenging because it requires long-horizon commitment under uncertainty and opponent interference.

3.3 TASK DESIGN PRINCIPLES

The task design is guided by comparison against the best legal alternative under the same runtime legality constraints as the game server. This ensures agents are not evaluated against illegal moves or overly penalized for valid second-best choices. For multi-factor decisions, we use component-weighted rubrics. Diversity and coverage are ensured by mapping all player actions to four categories: spatial placement, resource management and trading, opponent interaction, and strategic commitment. Difficulty is calibrated by reasoning horizon: easy tasks involve immediate tactical choices, medium tasks require turn-level planning across competing actions, and hard tasks require multi-turn or opening-phase reasoning where delayed effects dominate and local scoring can be misleading.

3.4 DIFFICULTY DISTRIBUTION

Difficulty was determined based on the decision horizon and the amount of strategic context required, as summarised in Table 1. Easy tasks are short-term decisions with immediate consequences and relatively direct legal alternatives. Medium tasks require turn-level prioritization across competing plans. Difficult tasks involve opening or multi-turn strategic commitments.

Category	Difficulty	Tasks
Short-Term Immediate tactical choices	Easy	settlement-location-selection; road-placement-direction; knight-card-playing-decision; year-of-plenty-card-playing-decision; monopoly-card-playing-decision; road-building-card-playing-decision; robber-victim-selection; robber-placement; accept-or-reject-trade-offers; generate-trade-offers; select-targeted-trade-partner; generate-counter-trade-offer
Medium-Term Turn-level prioritization	Medium	build-vs-save-decision; city-vs-settlement-vs-road-prioritization; development-card-purchase-decision; discard-strategy-after-seven; bank-trade-decision
Long-Term Opening and strategic race commitments	Difficult	initial-settlement-location-selection; decide-pursue-longest-road; decide-pursue-largest-army

Table 1: Difficulty distribution for the 20 CatanBench tasks.

4 EVALUATION PROTOCOL

Our evaluation metrics include win rate, average final victory points, average rounds to win, task success rate, average task score, and average latency per turn. A task is counted as successful when the selected action is legal and its score is at least the task-specific threshold listed in Appendix A.2. Before aggregation, all metrics are normalized to $[0, 1]$ with higher always better. Evaluation reports should include overall score, gameplay/reasoning/operations scores, win rate, average final victory points, average rounds to win, task success rate, average task score, per-task breakdowns, and average latency per turn. The overall score is: $\text{overallScore} = 0.30 \text{ winRate} + 0.15 \text{ averageFinalVictoryPoints}$

$$+ 0.15 \text{ averageRoundsToWin} + 0.30 \text{ taskSuccessRate} \\ + 0.10 \text{ averageLatencyPerTurn}.$$

5 BENCHMARK VALIDATION

We validated CatanBench using three forms of evidence: a human baseline (fig.1), agent baselines (fig 2), and task review (fig.3).



(a) Human baseline game state.

PLAYER STANDINGS						
1 games · 270 tasks · 251 logs						
RANK	NAME	SCORE	WIN	TASKS	VP	LATENCY
#1	VAYUN UNDERBORN	0.744	100.0%	77.5%	100	7021 MS
#2	SHARAYU UNDERBORN	0.302	0.0%	75.6%	50	15476 MS

(b) Human baseline leaderboard.

Figure 1: Human baseline results from the Vayun-Sharayu benchmark-enabled game. Task success rates are similar across players (77.5% vs. 75.6%), but benchmark scores differ due to high variance in win rate. As the number of games increases, win rates are expected to stabilize (e.g., 60-40 rather than 100-0), narrowing this gap.

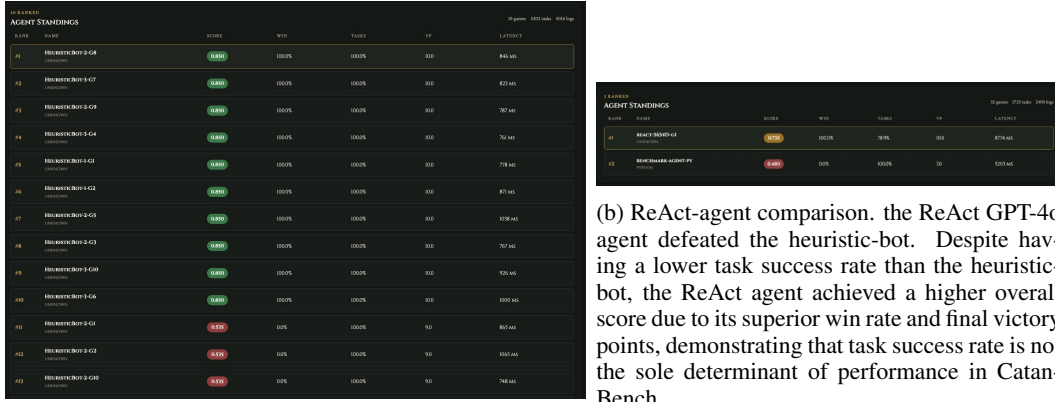


Figure 2: Agent baseline evidence.

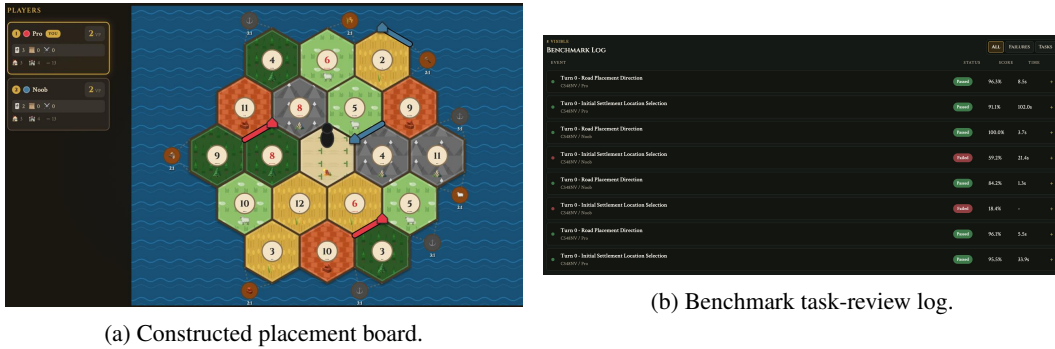


Figure 3: Task-review evidence for intentionally good and bad initial settlement placements. Strong initial placements (played by the red player) passed with scores such as 91.1% and 95.5%, while deliberately poor placements (played by the blue player) failed with scores such as 59.2% and 18.4%.

6 CONCLUSION

CatanBench is a real-time benchmark for strategic reasoning in Settlers of Catan. It evaluates 20 live gameplay tasks across tactical decisions, resource prioritization, negotiation, and long-horizon commitments. By scoring decisions against the best legal alternative at the moment of play, CatanBench provides denser and more diagnostic feedback than final win rate alone. We expect the benchmark to help researchers study agents that must reason under hidden information, stochastic outcomes, negotiation, and delayed strategic consequences. Future work includes larger human and agent baselines, stronger negotiation modeling, additional scenario suites, and calibration against expert Catan play.

REFERENCES

- Klaus Teuber. Catan base game rules. Catan Studio, originally published 1995. catan.com/sites/default/files/2021-06/catan_base_rules_2020_200707.pdf.
- Viral Doshi. Catan Online: A real-time multiplayer implementation of Settlers of Catan. GitHub repository, commit 3a0a6b815ff999adf5fd5802fa3df9b99833fc2d, January 1, 2026. github.com/Viral-Doshi/catan.

216 Anthony Costarelli, Mat Allen, Roman Hauksson, Grace Sodunke, Suhas Hariharan, Carlson Cheng,
217 Wenjie Li, and Arjun Yadav. GameBench: Evaluating strategic reasoning abilities of LLM agents.
218 *arXiv preprint arXiv:2406.06613v1*, June 7, 2024. arxiv.org/html/2406.06613v1.
219
220 Matan Halevy. Introducing CivBench Season 001. ClashAI, February 25, 2026. [clashai.live/](https://clashai.live/blog/ai/introducing-civbench-season-001)
221 [blog/ai/introducing-civbench-season-001](https://clashai.live/blog/ai/introducing-civbench-season-001).
222 Meg Risdal. Introducing Kaggle Game Arena. Kaggle, August 4, 2025. [https://www.kaggle.](https://www.kaggle.com/blog/introducing-game-arena)
223 [com/blog/introducing-game-arena](https://www.kaggle.com/blog/introducing-game-arena).
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269

A BENCHMARK TASK EXPLANATIONS

A.1 DESIGN PRINCIPLES

The benchmark uses three families of tasks:

- **Short-term tasks:** immediate tactical choices with direct board consequences.
- **Medium-term tasks:** action prioritization and resource conversion decisions.
- **Long-term tasks:** strategic race commitments whose value unfolds over several turns.

The main design goal is to compare decisions with the best legal alternative available at that moment (based on a heuristic approach), while keeping a threshold value to determine success or failure on the individual task. This system rewards near-optimal play when several choices are reasonable, and avoids placing fail labels on decisions that were reasonable but not perfectly aligned with the benchmark heuristic.

A.2 TASK SPECIFICATIONS

Each benchmark event is stored as a task record with a `taskId`, acting player, game snapshot, candidate-option context, selected action, scalar score, threshold, pass/fail label, and explanation. The expected output is always the agent’s selected legal action or explicit non-action. A task succeeds when the selected output is legal and its score meets or exceeds the threshold below; for option-ratio tasks, the score is the selected option’s value divided by the best legal option’s value, sometimes with the margin-aware adjustment described in the task explanation.

Task	Inputs	Expected output	Success criterion
initial-settlement-location-selection	Setup-phase board state, legal opening settlement vertices, adjacent hex resources/numbers, port metadata, expansion edges, acting player seat.	Chosen opening settlement vertex key.	Legal vertex and settlement-quality score ≥ 0.75 .
settlement-location-selection	Main/setup board state after an agent commits to settlement building, legal settlement vertices, adjacent production, resource diversity, port and expansion metadata.	Chosen settlement vertex key.	Legal vertex and selected-to-best settlement score ≥ 0.90 .
road-placement-direction	Current road network, legal candidate edges, endpoint occupancy, reachable settlement sites, opponent road/building pressure, Longest Road status.	Chosen road edge key.	Legal edge and road-placement score ≥ 0.80 .
knight-card-playing-decision	Player development cards, robber location, blocked production, resource-need map, player/opponent Knight counts, current phase legality.	Binary choice: play Knight now or hold.	Legal timing choice and play/hold option score ≥ 0.80 .
year-of-plenty-card-playing-decision	Player hand, all build-plan candidates, resource-need map, current affordable plans, legal Year of Plenty resource choices.	Two selected resources.	Legal resource pair and pair score ≥ 0.85 .
monopoly-card-playing-decision	Player hand, opponent resource holdings available to the server evaluator, build-plan table, resource-need map, legal Monopoly resource calls.	Selected resource to monopolize.	Legal resource call and resource score ≥ 0.85 .
road-building-card-playing-decision	Player road network, all legal road edges, road-placement scores, self/opponent road lengths, Longest Road status.	Binary choice: play Road Building now or hold.	Legal timing choice and play/hold option score ≥ 0.85 .
robber-victim-selection	Robber destination hex, eligible victims on that hex, public VP/achievement data, visible resource counts, turn order.	Selected victim player ID or no victim when none is legal.	Legal victim choice and victim option score ≥ 0.75 .
robber-placement	All legal robber hexes, current robber location, adjacent buildings, dice weights, public leader information, opponent hand sizes, self-harm flags.	Selected robber hex ID.	Legal hex and robber-placement score ≥ 0.75 .
accept-or-reject-trade-offers	Incoming offer/request resources, responder hand, proposer identity, visible leader status, responder build plans, need/surplus maps.	Binary response: accept or reject.	Legal response and response option score ≥ 0.95 ; accept is preferred only when trade quality is at least 0.70.
generate-trade-offers	Acting player’s hand, candidate offer/request bundles, target opponents, visible leader status, build plans, need/surplus maps.	Generated offer: give resources, request resources, and optional target.	Legal offer and trade-offer score ≥ 0.70 .
select-targeted-trade-partner	Candidate opponent list, requested resources, opponent availability estimate, visible leader status, trade self-gain context.	Selected target opponent ID.	Legal target and partner option score ≥ 0.90 .

Task	Inputs	Expected output	Success criterion
generate-counter-trade-offer	Incoming trade offer, responder hand, proposer identity, legal counteroffer bundles, responder build plans, need/surplus maps.	Counteroffer resources to give and request, or no counter when none is legal/useful.	Legal counteroffer and counter-trade score ≥ 0.70 .
build-vs-save-decision	Player hand, affordable and unaffordable road/settlement/city/development-card plans, current best future plan, seven-discard risk.	Build-family action or explicit save/no-build decision.	Legal selected action and spend/save option score ≥ 0.85 .
city-vs-settlement-vs-road-prioritization	Currently legal city upgrades, settlement vertices, normal road edges, build-family scores, current hand and board constraints.	Selected build family: city, settlement, road, or no legal build.	Legal build family and option score ≥ 0.90 .
development-card-purchase-decision	Player hand, development deck availability, held unplayed development cards, army-race context, competing non-dev build plans.	Binary choice: buy development card or skip.	Legal purchase/skip choice and option score ≥ 0.85 .
discard-strategy-after-seven	Player resource hand, required discard count, all legal discard bundles, retained-hand build-plan values.	Discard resource bundle.	Legal bundle with retained-hand value ratio ≥ 0.60 .
bank-trade-decision	Player hand, bank/port trade ratios, candidate give/get conversions, need/surplus maps, build unlocks, discard risk, no-trade option.	Bank/port trade conversion or no-trade decision.	Legal conversion/no-trade and option score ≥ 0.75 .
decide-pursue-longest-road	Snapshot of self and rival road lengths, Longest Road holder, visible VP, roads built so far, future road progress over the delayed horizon.	Strategic intent: pursue/contest Longest Road or defer.	Triggered only in real race states; delayed outcome score ≥ 0.30 .
decide-pursue-largest-army	Snapshot of self and rival Knights, Largest Army holder, visible VP, hand size, development cards bought, future Knight/army progress over the delayed horizon.	Strategic intent: pursue/contest Largest Army or defer.	Triggered only in real army-race states; delayed outcome score ≥ 0.30 .

A.3 SHARED HEURISTIC BUILDING BLOCKS

A.3.1 PRODUCTION EXPECTANCY

Some dice numbers have a higher probability of being rolled than others. Specifically, the probability weights are:

$$2, 12 \mapsto 1, \quad 3, 11 \mapsto 2, \quad 4, 10 \mapsto 3, \quad 5, 9 \mapsto 4, \quad 6, 8 \mapsto 5.$$

7 is a number used for the robber and does not produce resources. To compute the expected value of the resources gained on the next turn from a current settlement vertex,

$$\begin{aligned} \text{productionWeight} &= \sum \text{adjacent_hex_probability_weights}, \\ \text{resourceProductionExpectancy} &= \min \left(1, \frac{\text{productionWeight}}{15} \right). \end{aligned}$$

The denominator 15 is the theoretical maximum because a vertex touches at most three hexes and the maximum per-hex weight is 5.

A.3.2 RESOURCE DIVERSITY

Each settlement vertex is adjacent to up to three hexes, which can produce up to three different resource types (`distinct_adjacent_resource_types`).

Resource diversity is defined as

$$\text{resourceDiversity} = \min \left(1, \frac{\text{distinct_adjacent_resource_types}}{3} \right).$$

It matters because diverse starts reduce dependency on trading, improve build robustness, and lower the risk of bottlenecking on a single missing resource.

A.3.3 EXPANSION ACCESS

In crowded boards, some settlement locations have more future legal road placements than others, which translates into better expansion potential and better survivability in case of unfavorable dice rolls or robber placements. For a settlement candidate, the evaluator computes incident edges, keeps only edges where a road could legally be placed, and defines

$$\text{expansionAccess} = \min \left(1, \frac{\text{legal_outgoing_edges}}{3} \right).$$

A.3.4 PORT SYNERGY

We use a discrete port heuristic:

- specific 2:1 port $\rightarrow 1.0$
- generic 3:1 port $\rightarrow 0.7$
- no port $\rightarrow 0.0$

Ports matter because they convert surplus resources into missing ones; specific ports are usually stronger due to higher exchange efficiency.

A.3.5 BUILD PLAN BASE SCORES

Several tasks construct plan candidates for `road`, `settlement`, `city`, and `devCard`. Each plan has a `baseScore`: the current heuristic value of that build family before applying affordability or distance-to-completion discounts. The shared definitions are:

- `road`: the maximum score among currently legal road placements under the road-placement rubric from Section A.4.3.
- `settlement`: the maximum score among currently legal settlement placements under the settlement rubric from Section A.4.2.
- `city`: the maximum score among legal city upgrades using $0.65 \cdot \text{production} + 0.10 \cdot \text{diversity} + 0.05 \cdot \text{portSynergy} + 0.20$.
- `devCard`: 0 if the development-card deck is unavailable; otherwise

$$\begin{aligned} \text{devCardBaseScore} = & \text{clamp}(\max(0.15, \\ & 0.45 \cdot (0.5 + 0.5 \cdot \text{cityPlanScore}) + \text{armyRaceBonus} \\ & - \text{devCardCongestionPenalty} \\ & - \text{expansionPressurePenalty})). \end{aligned}$$

Here `armyRaceBonus` is 0.20 when another development-card purchase could plausibly complete or contest Largest Army; `devCardCongestionPenalty` increases with held unplayed development cards and is 0.35 at three or more held cards; and `expansionPressurePenalty` is 0.18 when road, settlement, or city plans have strong current value. This prevents the evaluator from treating repeated card buying as optimal while the player is sitting on playable cards or clear board-development opportunities.

A.3.6 RESOURCE NEED AND SURPLUS

Resource need and surplus are used in trade and bank-trade scoring.

For each candidate plan and resource r , `cost[r]` is the number of cards of resource r required by the plan, `hand[r]` is the player's current count of resource r , and

$$\text{missing}[r] = \max(0, \text{cost}[r] - \text{hand}[r]).$$

`totalMissing` is the total number of resource cards still needed for the plan:

$$\text{totalMissing} = \sum_r \text{missing}[r].$$

Each plan is weighted according to how many resources are needed to achieve the plan:

$$\text{planWeight} = \frac{\text{baseScore} \cdot \text{priorityMultiplier}}{1 + 0.75 \cdot \text{totalMissing}},$$

where `baseScore` uses the shared build-plan definition from Section A.3.5. The current priority multipliers are 1.5 for settlement, 1.1 for city, 0.75 for devCard, and 0.25 for road. These multipliers intentionally make settlement-unlocking resources such as wool, grain, brick, or lumber more valuable than extra road resources when a legal expansion site exists. For each resource r ,

$$\text{need}[r] += \text{planWeight} \cdot \frac{\text{missing}[r]}{\max(1, \text{totalMissing})}.$$

For surplus, the evaluator reserves part of the current hand for weighted plans:

$$\text{reserved}[r] = \min(\text{hand}[r], \sum \text{planWeight} \cdot \text{cost}[r]),$$

then treats any unreserved cards of resource r as surplus. The surplus map is normalized by the largest unreserved resource count in the hand:

$$\text{surplus}[r] = \frac{\max(0, \text{hand}[r] - \text{reserved}[r])}{\max(1, \max_s \max(0, \text{hand}[s] - \text{reserved}[s]))},$$

where s ranges over all resource types. This avoids earlier failure modes where a resource such as ore looked important merely because it was scarce in hand, even when the board strongly favored early road and settlement expansion.

A.4 TASK-BY-TASK EXPLANATIONS

A.4.1 INITIAL-SETTLEMENT-LOCATION-SELECTION

$$\begin{aligned} \text{Score} = & 0.40 \cdot \text{production} + 0.25 \cdot \text{diversity} \\ & + 0.20 \cdot \text{expansionAccess} + 0.15 \cdot \text{portSynergy}. \end{aligned}$$

This rubric uses the shared `production`, `diversity`, `expansionAccess`, and `portSynergy` definitions from Sections A.3.1–A.3.4. Production receives the highest weight because early income determines the opening trajectory, while the other terms preserve resource mix, expansion geometry, and port upside. The evaluator applies the rubric above to the shared settlement-quality quantities and clamps the final score to $[0, 1]$.

The threshold for `initial-settlement-location-selection` is 0.75 because opening settlement quality is highly consequential but multiple strong starts can be strategically defensible.

A.4.2 SETTLEMENT-LOCATION-SELECTION

This task uses the same scoring system as the initial-settlement task in Section A.4.1. The core difference between initial and subsequent location selection is when the task is emitted and how it is interpreted. The initial settlement task is a long-term opening decision with a lower pass threshold, while subsequent settlement placement is a short-term task evaluated against a smaller set of legal options. This reflects the fact that the first settlement sets the foundation for the entire game and is a critical decision point, while subsequent settlement placements, although still important, are often more constrained by the existing board state.

The threshold for `settlement-location-selection` is 0.90 because later settlement choices are evaluated against a smaller legal option set and should closely match the best available site.

A.4.3 ROAD-PLACEMENT-DIRECTION

$$\begin{aligned} \text{Score} = & 0.25 \cdot \text{futureReachability} + 0.20 \cdot \text{reachableValue} \\ & + 0.10 \cdot \text{blockingValue} + 0.25 \cdot \text{settlementTargetValue} \\ & + 0.15 \cdot \text{extensionValue} + 0.05 \cdot \text{cycleAvoidance}. \end{aligned}$$

Where:

- `futureReachability` is the forward vertex's onward legal-road count, clamped after division by 3. In main-phase road scoring, the forward vertex is the newly opened endpoint when the candidate road has one; only if both endpoints are already part of the player's network does the evaluator fall back to the better endpoint.
- `reachableValue` is the forward vertex's weighted settlement value:

$$\text{reachableValue} = 0.5 \cdot \text{production} + 0.3 \cdot \text{diversity} + 0.2 \cdot \text{portSynergy}.$$
- `blockingValue` is the maximum blocking score over the candidate road's two endpoints and all opponents present at those endpoints.

- `settlementTargetValue` is the best open settlement value at either endpoint of the road, using the same production-diversity-port mixture as `reachableValue`. Endpoints occupied by a building, invalidated by the distance rule, or already incident to one of the acting player's roads receive 0 because they do not create a new settlement frontier.
- `extensionValue` rewards roads that extend a road network outward. It is 1 when the new road creates an open endpoint while touching an existing own building or road, 0.6 for weaker outward extensions, and 0 when it does not create meaningful outward reach.
- `cycleAvoidance` is 0 when the candidate road closes back into the player's existing road network at both endpoints, and 1 otherwise. This discourages small loops that consume brick and lumber without creating new settlement access.

When the acting player already has a legal connected settlement target, the evaluator sets `settlementTargetValue` and `extensionValue` for additional main-phase roads to 0. In that state, extra roads are treated as a distraction from converting the existing expansion point into a victory point unless they provide exceptional tactical blocking value. For each endpoint-opponent pair, the evaluator ignores endpoints occupied by the acting player and endpoints occupied by a different opponent. It then computes:

$O_{\text{branch}} = \text{open incident edges excluding the candidate road,}$

$$\text{branchScarcity} = \text{clamp}\left(\frac{2 - O_{\text{branch}}}{2}\right),$$

$$\text{vertexOpportunity} = \begin{cases} 0.8, & \text{if the opponent already owns the endpoint,} \\ 1.0, & \text{if the endpoint is an open legal settlement site,} \\ 0.35, & \text{otherwise} \end{cases}$$

Let R_{adj} be the number of opponent roads adjacent to the endpoint, B_{opp} be 1 if the opponent owns the endpoint and 0 otherwise, L_{opp} be the opponent's road length, and H_{opp} be 1 if the opponent has Longest Road and 0 otherwise.

$$\text{presenceStrength} = \text{clamp}\left(\frac{R_{\text{adj}} + B_{\text{opp}}}{2}\right),$$

$$\text{roadPressure} = \text{clamp}\left(\frac{L_{\text{opp}} + 2 \cdot H_{\text{opp}}}{8}\right).$$

The candidate blocking score is

$$\text{candidateBlocking} = \text{clamp}(0.50 \cdot \text{branchScarcity} + 0.30 \cdot \text{vertexOpportunity} + 0.10 \cdot \text{presenceStrength} + 0.10 \cdot \text{roadPressure}).$$

and `blockingValue` is the largest `candidateBlocking`, or 0 if no opponent is present at either endpoint. The current implementation therefore recognizes common denial plays without running a full tactical search. The updated road rubric treats roads primarily as instruments for future victory points: settlement-target value and outward extension are weighted alongside reachability, while cycle avoidance penalizes roads that form small loops without opening new build sites. Blocking remains useful, but it is deliberately secondary because most road purchases should either unlock settlements or advance a real Longest Road plan.

The threshold for `road-placement-direction` is 0.80 because roads are path-dependent investment moves and several near-best routes can preserve similar future options.

A.4.4 KNIGHT-CARD-PLAYING-DECISION

Define:

- `robberThreatLevel` measures how costly the current robber placement is for the player.
- `resourceNeed` is the largest entry in the goal-aware resource-need map from Section A.3.6.

- `armyProgressRatio` measures progress toward Largest Army after playing the Knight:

$$\text{robberThreatLevel} = \begin{cases} \text{clamp}(\text{robberNumberWeight}/5), & \text{if blocked} \\ 0.15, & \text{otherwise} \end{cases}$$

$$\text{resourceNeed} = \max_r \text{need}[r].$$

This captures how valuable a Knight steal is when the player is missing an important resource for a near-term plan. Let K_{self} be the player's current Knight count and K_{opp} be the largest opponent Knight count. Then:

$$\text{armyProgressRatio} = \text{clamp}\left(\frac{K_{\text{self}} + 1}{\max(3, K_{\text{opp}} + 1)}\right).$$

Playing a Knight is most valuable when the robber is harming the player, when stealing a needed resource is likely to matter, and when Largest Army progress matters. The score does not include resource hand size because the Knight card has already been purchased and playing it does not reduce seven-risk for the player's resource hand.

The live scores are

$$\text{playNowScore} = 0.45 \cdot \text{robberThreatLevel} + 0.35 \cdot \text{resourceNeed} + 0.20 \cdot \text{armyProgressRatio}$$

$$\text{holdScore} = \text{clamp}(0.25 + 0.75 \cdot (1 - \text{playNowScore})).$$

The threshold for knight-card-playing-decision is 0.80 because Knight timing often has several defensible play-now versus hold states in live games. This task uses the larger of the selected-to-best ratio and a margin-aware score $1 - (\text{bestScore} - \text{selectedScore})$ so that near-tied timing choices do not count as major heuristic failures.

A.4.5 BUILD-VS-SAVE-DECISION

This decision compares the expected value of spending now against preserving option value for later, incorporating immediate gains, future unlock potential, and seven-risk. The runtime constructs plan candidates for road, settlement, city, and devCard. Each plan uses the shared `baseScore` definitions from Section A.3.5, a cost vector, and a `missingCount`. `missingCount` is the total number of missing resource cards for the plan's cost vector. The discounted future value is

$$\text{effectiveScore} = \frac{\text{baseScore}}{1 + 0.75 \cdot \text{missingCount}}.$$

Each affordable build option gets `score = clamp(baseScore)`. The save alternative is

$$B_{\text{eff}} = \text{best unaffordable effectiveScore}.$$

$$\text{saveScore} = \text{clamp}\left(\max(0.20, B_{\text{eff}} - P_7)\right),$$

where

$$P_7 = \text{clamp}(0.08 \cdot \max(0, \text{resourceTotal} - 7)).$$

This penalizes saving large hands that are exposed to robber discard risk.

The threshold for build-vs-save-decision is 0.85 because spend/save choices compare broad strategic options and often involve several near-equivalent actions. This task uses the larger of the selected-to-best ratio and a margin-aware score $1 - (\text{bestScore} - \text{selectedScore})$ so that near-tied low-confidence build options are not treated as major failures.

A.4.6 YEAR-OF-PLENTY-CARD-PLAYING-DECISION

Year of Plenty is strongest when it immediately unlocks a valuable build, repairs bottlenecks, and improves future resource flexibility. The runtime waits until both chosen resources are known, then scores every unordered resource pair using the shared plan table from Section A.3.5. It computes `beforeBest` as the largest `baseScore` affordable before the card, or 0 if no plan is affordable. It then simulates each candidate resource pair and computes `afterBest` the same way from the simulated post-card hand. For each pair:

$$\text{buildUnlock} = \text{clamp}(\max(0, \text{afterBest} - \text{beforeBest}) + U_{\text{imm}}),$$

where U_{imm} is `afterBest` if `afterBest` > `beforeBest` and 0 otherwise.

$$\text{resourceScarcity} = \text{clamp}\left(\frac{\text{need}[r_1] + \text{need}[r_2]}{2}\right),$$

where r_1 and r_2 are the two resources in the candidate pair.

$$\text{longtermDiversityGain} = \text{clamp}\left(\frac{\text{distinctAfter} - \text{distinctBefore}}{2}\right).$$

The final score is

$$\text{Score} = \text{clamp}(0.50 \cdot \text{buildUnlock} + 0.35 \cdot \text{resourceScarcity} + 0.15 \cdot \text{longtermDiversityGain}).$$

The threshold for `year-of-plenty-card-playing-decision` is 0.85 because multiple resource pairs can be strategically close when they unlock similar plans or preserve similar flexibility. This task uses the larger of the `selected-to-best` ratio and a margin-aware score $1 - (\text{bestScore} - \text{selectedScore})$ so that near-equivalent pairs do not cause the calibration heuristic to fail its own benchmark.

A.4.7 MONOPOLY-CARD-PLAYING-DECISION

Monopoly is strongest when opponents likely hold a concentrated resource, the acting player currently needs that resource, and the resulting swing unlocks a build. For each candidate called resource, the server sums opponents' actual holdings of that resource, computes `beforeBest`, simulates adding all stolen copies, and recomputes `afterBest`. It then uses:

$$\text{estimatedOpponentHoldings} = \text{clamp}\left(\frac{C_r}{C_{\text{max}}}\right),$$

where C_r is the total opponent copies of the called resource and C_{max} is the largest opponent holding count across any resource.

$$\text{personalResourceNeed} = \text{clamp}(\text{need}[r]).$$

$$\text{buildUnlock} = \text{clamp}(\max(0, \text{afterBest} - \text{beforeBest}) + U_{\text{imm}}),$$

where U_{imm} is `afterBest` if `afterBest` > `beforeBest` and 0 otherwise. The resource score is

$$\text{resourceScore} = \text{clamp}(0.55 \cdot \text{estimatedOpponentHoldings} + 0.30 \cdot \text{personalResourceNeed} + 0.15 \cdot \text{buildUnlock}).$$

Monopoly plays emit a benchmark payload over the five possible resource calls. The implementation uses actual hidden-opponent holdings available on the server.

The threshold for `monopoly-card-playing-decision` is 0.85 because several resource calls can be strategically close when opponent holdings and personal build unlocks point in slightly different directions. This task uses the larger of the `selected-to-best` ratio and a margin-aware score $1 - (\text{bestScore} - \text{selectedScore})$ so that near-equivalent calls do not over-penalize the reference heuristic.

A.4.8 ROAD-BUILDING-CARD-PLAYING-DECISION

The central question is whether playing Road Building now is more valuable than holding it for a better timing window. The evaluator scores every currently legal road placement using the road-placement heuristic from Section A.4.3, sorts scores descending, averages the best two legal road scores (or one if only one exists), and computes

$$\text{playNowScore} = \text{clamp}(0.70 \cdot \text{topTwoAverage} + 0.30 \cdot \text{longestRoadLeverage}),$$

where

$$\text{longestRoadLeverage} = \text{clamp}\left(\frac{(L_{\text{self}} + 2) - L_{\text{opp}} + 3}{6}\right).$$

Here L_{self} is the player's current road length and L_{opp} is the best opponent road length. The hold alternative is

$$\text{holdScore} = \text{clamp}(0.30 + 0.70 \cdot (1 - \text{playNowScore})).$$

The threshold for road-building-card-playing-decision is 0.85 because Road Building timing has tactical flexibility and a slightly wider range of good play-now choices.

A.4.9 ROBBER-VICTIM-SELECTION

This task is evaluated as an option ratio over legal victims. Victim selection is a discrete choice whose value depends on leader pressure, theft value, and visible board strength. The visible leader is the player with the highest public score:

$$\text{visibleLeaderScore} = \text{visibleVP} + 2 \cdot \text{hasLongestRoad} + 2 \cdot \text{hasLargestArmy}.$$

If multiple players tie, the implementation keeps the first tied player in turn-order/index order. For each legal victim v , the score is

$$\text{victimScore}(v) = \text{clamp}(\text{leaderBase}(v) + \text{handValue}(v) + \text{visibleStrength}(v)),$$

where

$$\text{leaderBase}(v) = \begin{cases} 0.55, & \text{if } v \text{ is the visible leader,} \\ 0.25, & \text{otherwise,} \end{cases}$$

$$\text{handValue}(v) = \min\left(0.35, \frac{\text{resourceTotal}(v)}{20}\right),$$

$$\text{visibleStrength}(v) = \min\left(0.10, \frac{\text{visibleScore}(v)}{20}\right).$$

The threshold for robber-victim-selection is 0.75 because legal victims on the chosen hex often differ only modestly in visible hand value and leader pressure. This task uses the larger of the selected-to-best ratio and a margin-aware score $1 - (\text{bestScore} - \text{selectedScore})$ so that close victim choices do not over-penalize the calibration heuristic.

A.4.10 ROBBER-PLACEMENT

$$\text{Score} = 0.35 \cdot \text{hurtLeader} + 0.30 \cdot \text{productionDamage} + 0.20 \cdot \text{theftValue} + 0.15 \cdot \text{selfHarmAvoidance}.$$

For a candidate hex h , the evaluator uses the same visible leader definition from Section A.4.9. It identifies all opponents on that hex and computes each opponent's strength proxy:

$$\text{strength}(p) = \min\left(1, \frac{\text{visibleVP}(p) + \text{resourceTotal}(p)/8}{5}\right).$$

Let $w(h)$ be the dice-number probability weight of the hex. The damage terms are:

$$\text{hurtLeader} = \min\left(1, \frac{w(h) \sum_{p \in h, p = \text{visible leader}} \text{strength}(p)}{3}\right),$$

$$\begin{aligned} \text{productionDamage} &= \min \left(1, \frac{w(h) \sum_{p \in h, p \neq \text{self}} \text{strength}(p)}{5} \right), \\ \text{theftValue} &= \max_{p \in h, p \neq \text{self}} \min \left(1, \frac{\text{resourceTotal}(p)}{7} \right), \\ \text{selfHarmAvoidance} &= \begin{cases} 0, & \text{if the acting player has a building on } h, \\ 1, & \text{otherwise.} \end{cases} \end{aligned}$$

Hurting the leader is the anti-runaway term, production damage captures future income suppression, theft value captures immediate swing, and self-harm avoidance prevents obvious placements on the acting player's own production.

The threshold for robber-placement is 0.75 because robber placement is tactical but often has several near-equivalent hexes that trade off leader pressure, production denial, theft value, and self-harm. This task uses the larger of the selected-to-best ratio and a margin-aware score $1 - (\text{bestScore} - \text{selectedScore})$ so that tiny heuristic differences between strong hexes do not penalize the reference heuristic agent.

A.4.11 ACCEPT-OR-REJECT-TRADE-OFFERS

$$\begin{aligned} \text{tradeQuality} &= 0.35 \cdot \text{legality} + 0.45 \cdot \text{selfGain} \\ &\quad + 0.15 \cdot (1 - \text{leaderPenalty}) + 0.05 \cdot (1 - \text{fairnessPenalty}). \end{aligned}$$

The response task uses the same plan-aware trade-quality score as generated trade offers, but from the responder's perspective. The responder should accept iff the incoming offer's `tradeQuality` is at least 0.70; otherwise rejection receives the benchmark-best option score. This makes the response task consistent with the offer-generation task and avoids penalizing settlement-unlocking trades that have low immediate surplus value but improve the next build plan.

The plan-aware self-gain term is:

$$\begin{aligned} \text{selfGain} &= 0.45 \cdot \text{requestValue} + 0.35 \cdot \text{planGain} \\ &\quad + 0.20 \cdot \text{settlementGain} - 0.35 \cdot \text{giveCost}, \end{aligned}$$

clamped to $[0, 1]$. Here `requestValue` uses the responder's goal-aware need map, `giveCost` uses the responder's surplus map, `planGain` is the improvement in the best affordable weighted build plan after the trade, and `settlementGain` is the reduction in missing settlement resources when a legal settlement target exists.

The threshold for `accept-or-reject-trade-offers` is 0.95 because accepting or rejecting a concrete offer should be nearly optimal to avoid rewarding harmful trades.

A.4.12 GENERATE-TRADE-OFFERS

$$\begin{aligned} \text{tradeOfferScore} &= 0.35 \cdot \text{legality} + 0.45 \cdot \text{selfGain} \\ &\quad + 0.15 \cdot (1 - \text{leaderPenalty}) + 0.05 \cdot (1 - \text{fairnessPenalty}). \end{aligned}$$

A trade must first be legal. Its main purpose is to improve the acting player's position. Helping the leader is strategically risky, and highly unfair offers are less plausible and less likely to be accepted. The components are:

$$\begin{aligned} \text{selfGain} &= \text{clamp}(0.45 \cdot \text{requestValue} + 0.35 \cdot \text{planGain} \\ &\quad + 0.20 \cdot \text{settlementGain} - 0.35 \cdot \text{giveCost}), \\ \text{fairnessPenalty} &= \min(1, \max(0, \text{giveCost} - \text{requestValue})), \\ \text{legality} &= \begin{cases} 0, & \text{if the offer gives resources the player does not own,} \\ 1, & \text{otherwise,} \end{cases} \\ \text{leaderPenalty} &= \begin{cases} 1, & \text{if the target is the visible leader,} \\ 0, & \text{otherwise.} \end{cases} \end{aligned}$$

The generated offer is compared against the benchmark-best score via a ratio.

The threshold for `generate-trade-offers` is 0.70 because generated offers are exploratory and many legal, plausible offers may be useful even if not benchmark-best.

A.4.13 SELECT-TARGETED-TRADE-PARTNER

This task is evaluated as an option ratio. Each candidate partner is scored as

$$\text{partnerScore} = \text{selfGain} + \text{availabilityBonus} - 0.4 \cdot \text{leaderHelpPenalty},$$

where

$$\text{availabilityBonus} = \begin{cases} 0.25, & \text{if the partner can provide the requested resources,} \\ 0, & \text{otherwise,} \end{cases}$$

$$\text{leaderHelpPenalty} = \begin{cases} 1, & \text{if the partner is the visible leader,} \\ 0, & \text{otherwise.} \end{cases}$$

This favors partners that produce good value and can satisfy the request, while discounting trades with the leader. Targeted trade proposals emit this task at runtime, and the selected target is compared against all opponent candidates rather than only against itself.

The threshold for `select-targeted-trade-partner` is 0.90 because targeted trade partner choice should closely match availability, self-gain, and leader-avoidance signals.

A.4.14 GENERATE-COUNTER-TRADE-OFFER

$$\begin{aligned} \text{counterTradeScore} = & 0.35 \cdot \text{legality} + 0.45 \cdot \text{selfGain} \\ & + 0.15 \cdot (1 - \text{leaderPenalty}) \\ & + 0.05 \cdot (1 - \text{fairnessPenalty}). \end{aligned}$$

The structural reasoning follows the generated-trade-offer task from Section A.4.12, but now from the responder's perspective relative to an incoming offer. The payload reuses the same plan-aware trade-quality subscores as Section A.4.12:

$$\begin{aligned} \text{selfGain} = & \text{clamp}(0.45 \cdot \text{requestValue} + 0.35 \cdot \text{planGain} \\ & + 0.20 \cdot \text{settlementGain} - 0.35 \cdot \text{giveCost}), \\ \text{fairnessPenalty} = & \min(1, \max(0, \text{giveCost} - \text{requestValue})), \\ \text{legality} = & \begin{cases} 0, & \text{if the responder gives resources they do not own,} \\ 1, & \text{otherwise,} \end{cases} \\ \text{leaderPenalty} = & \begin{cases} 1, & \text{if the original proposer is the visible leader,} \\ 0, & \text{otherwise.} \end{cases} \end{aligned}$$

The benchmark payload stores this through `offerContext` with `bestOfferScore = 1` rather than logging each candidate counteroffer separately.

The threshold for `generate-counter-trade-offer` is 0.70 because counteroffers are exploratory like generated offers and should allow a broader set of reasonable proposals.

A.4.15 CITY-VS-SETTLEMENT-VS-ROAD-PRIORITIZATION

This task is evaluated as an option ratio. It compares build classes rather than board coordinates and asks which legal build family has the highest strategic value at the current state. Let the maxima below range over the currently legal options:

$$\begin{aligned} \text{settlementScore} = & \max \text{settlementScore}(v), \\ \text{roadScore} = & \max \text{roadScore}(e), \end{aligned}$$

where settlement scores use Section A.4.2 and road scores use Section A.4.3.

$$\text{cityScore} = 0.65 \cdot \text{production} + 0.10 \cdot \text{diversity} + 0.05 \cdot \text{portSynergy} + 0.20.$$

Production dominates because a city doubles an existing production site; diversity and port access still matter because they determine how valuable that amplification is, and the constant 0.20 reflects the guaranteed VP and output increase. The task is emitted for settlement, city, and normal road-building choices. Free roads from Road Building are intentionally excluded.

The threshold for `city-vs-settlement-vs-road-prioritization` is 0.90 because build-family prioritization should usually select the strongest currently legal development path.

A.4.16 DEVELOPMENT-CARD-PURCHASE-DECISION

This task is evaluated as an option ratio. Buying a development card is an investment choice against competing uses of resources and should be favored when hidden VP, Knight access, or tactical flexibility outweigh immediate board development. This decision is derived from the shared devCard plan baseScore from Section A.3.5:

$$\begin{aligned} \text{devCardBaseScore} = & \text{clamp}(\max(0.15, \\ & 0.45 \cdot (0.5 + 0.5 \cdot \text{cityPlanScore}) + \text{armyRaceBonus} \\ & - \text{devCardCongestionPenalty} \\ & - \text{expansionPressurePenalty})) \end{aligned}$$

if the development deck is available, otherwise 0. The congestion and expansion penalties are the same as those in Section A.3.5. The two options are

$$\begin{aligned} \text{buy-dev-card} &= \text{clamp}(\text{devCardBaseScore}), \\ \text{skip-dev-card} &= \text{clamp}(\max(\text{saveScore}, B_{\text{nondev}})), \end{aligned}$$

where B_{nondev} is the best affordable non-development-card plan baseScore. buyDevCard emits this task directly. If development-card purchase was affordable but skipped for the full turn, the negative case is recorded at turn end.

The threshold for development-card-purchase-decision is 0.85 because development-card purchases frequently compete with saving or other modest-value plans in low-confidence states. This task uses the larger of the selected-to-best ratio and a margin-aware score $1 - (\text{bestScore} - \text{selectedScore})$ so that near-tied spend decisions do not cause the calibration heuristic to fail its own benchmark.

A.4.17 DISCARD-STRATEGY-AFTER-SEVEN

This task is scored as

$$\frac{\text{RetainedHandValue}}{\text{BestRetainedValue}}.$$

Discarding is constrained by a fixed number of cards to lose, and the actual strategic objective is to maximize the value of the retained hand rather than the discarded hand. The evaluator enumerates all legal discard bundles summing to $\lfloor \text{total_hand_size}/2 \rfloor$, subtracts each from the current hand, and scores the retained hand with:

$$\begin{aligned} \text{bestAffordable} &= \max \text{ affordable plan baseScore,} \\ \text{bestFuture} &= \max \text{ plan effectiveScore,} \\ \text{diversity} &= \text{clamp} \left(\frac{\text{distinctPositiveResources}}{5} \right). \\ \text{retainedHandValue} &= \text{clamp}(0.55 \cdot \text{bestAffordable} + 0.30 \cdot \text{bestFuture} \\ &\quad + 0.15 \cdot \text{diversity}). \end{aligned}$$

The final task score is the selected retained-hand value divided by the best legal retained-hand value. Candidate discard bundles are generated directly from the player hand and required discard count. The live calibration agent handles discard before checking whether it is the current turn, because a player may be forced to discard off-turn when another player rolls a seven.

The threshold for discard-strategy-after-seven is 0.60 because discard choices are fully enumerated but the retained-hand heuristic compresses many strategically similar bundles into a fairly narrow score range. The calibration agent queries the server for the same best retained-hand bundle used by the benchmark scorer so that successful discard actions follow the benchmark's own preferred choice.

A.4.18 BANK-TRADE-DECISION

$$0.50 \cdot \text{resourceValueDelta} + 0.30 \cdot \text{buildUnlock} + 0.20 \cdot \text{opportunityCost}.$$

This task is currently approximated through a choice ratio over candidate bank/port trades plus a no-trade option. For each candidate trade, the evaluator computes the trade ratio using `GameLogic.getTradeRatio` and scores the candidate as

$\text{tradeScore} = \text{clamp}(\text{need}[\text{get}] - 0.5 \cdot \text{surplus}[\text{give}] + 0.5 + \text{discardRelief}),$
and includes a fixed baseline

$$\text{no-trade} = 0.45 - P_7,$$

where P_7 is the same above-seven-card discard-risk penalty used in build-vs-save-decision, and

$$\text{discardRelief} = \max(0, P_7^{\text{before}} - P_7^{\text{after}}).$$

Acquiring a needed resource is valuable, paying with a surplus resource is less painful than paying with a scarce one, and the no-trade baseline prevents weak legal trades from looking attractive by default. When a player is above seven cards, the heuristic also rewards trades that reduce discard exposure and makes no-trade less attractive. The calibration agent uses this same score when choosing which bank or port trade to execute, so a successful calibration-agent bank trade should select the benchmark-best available conversion under the current need and surplus maps. The named rubric components are not yet separately logged; they are compressed into a single trade desirability heuristic.

The threshold for bank-trade-decision is 0.75 because bank trades are deterministic but may have several near-equivalent resource-conversion options, especially when the current need and surplus maps are flat. This task uses the larger of the selected-to-best ratio and a margin-aware score $1 - (\text{bestScore} - \text{selectedScore})$ so that near-tied conversions and the no-trade baseline do not cause the calibration heuristic to fail its own benchmark.

A.4.19 DECIDE-PURSUE-LONGEST-ROAD

$$\begin{aligned} \text{legacyScore} = & 0.40 \cdot \text{roadProgressRatio} + 0.30 \cdot \text{opponentRoadGap} \\ & + 0.20 \cdot \text{resourceAffordability} + 0.10 \cdot \text{VPImpact}. \end{aligned}$$

The stronger formulation is to record a pending task episode, wait 4–6 turns for the evaluated player, and score using snapshot-plus-outcome evidence:

$$\begin{aligned} \text{pursueScore} = & 0.45 \cdot \text{blockingEffectiveness} \\ & + 0.30 \cdot \text{opponentProgressThreat} + 0.15 \cdot \text{selfRoadSynergy} \\ & + 0.10 \cdot \text{resourceCost}. \end{aligned}$$

This redesign is preferable because Longest Road decisions are not fully observable from a single action. The live system already uses the delayed v2-style episode. At decision time it snapshots player road length, strongest opponent road rival and their road length, whether either player currently holds Longest Road, player visible score, and roads built so far in benchmark long-term stats. To avoid treating ordinary expansion roads as Longest Road commitments, the task is only instantiated when there is an active race signal: either player already holds Longest Road, the player has at least six road length, the strongest rival has at least seven road length, or the player has at least five road length while close to the rival with a meaningful road plan.

After a four-turn horizon, decisive resolution, or game end, it computes:

$$G_{\text{self}} = \max(0, L_{\text{self, end}} - L_{\text{self, start}}), \quad G_{\text{target}} = \max(0, L_{\text{target, end}} - L_{\text{target, start}}),$$

$$R_{\Delta} = \max(0, \text{roadsBuiltSinceSnapshot}),$$

$$\text{opponentProgressThreat} = \text{clamp}\left(\frac{(L_{\text{target, start}} - L_{\text{self, start}}) + 2H_{\text{target, start}} + 2}{8}\right),$$

$$\text{blockingEffectiveness} = \text{clamp}(0.45 + 0.18G_{\text{self}} - 0.12G_{\text{target}} + 0.25H_{\text{self, end}}),$$

$$\text{selfRoadSynergy} = \text{clamp}\left(\frac{G_{\text{self}}}{4} + 0.35H_{\text{self, end}}\right).$$

Here each H indicator is 1 when that player has Longest Road at the named time and 0 otherwise. The resource-cost term is

$$\text{resourceCost} = \begin{cases} 1, & \text{successful defer and } R_{\Delta} = 0, \\ 0.5, & \text{pursue and } R_{\Delta} = 0, \\ \text{clamp}\left(\frac{G_{\text{self}} + \text{longestRoadBonus}}{R_{\Delta}}\right), & \text{otherwise.} \end{cases}$$

The final alternatives are

$$\begin{aligned} \text{pursueScore} = & \text{clamp}(0.45 \cdot \text{blockingEffectiveness} \\ & + 0.30 \cdot \text{opponentProgressThreat} + 0.15 \cdot \text{selfRoadSynergy} \\ & + 0.10 \cdot \text{resourceCost}), \end{aligned}$$

$$\begin{aligned} \text{deferScore} = & \text{clamp}(0.45 \cdot (1 - \text{opponentProgressThreat}) \\ & + 0.25 \cdot \left(1 - \min\left(1, \frac{G_{\text{target}}}{3}\right)\right) \\ & + 0.15 \cdot \left(1 - \min\left(1, \frac{G_{\text{self}}}{3}\right)\right) + 0.15D_0). \end{aligned}$$

Here D_0 is 1 if $R_{\Delta} = 0$ and 0.2 otherwise. This task now runs as a pending long-horizon episode, and finalization writes one ordinary benchmark task result after the horizon, decisive resolution, or game end.

The threshold for decide-pursue-longest-road is 0.30 because long-horizon race commitments are scored with delayed outcome evidence and can be affected by dice, opponent moves, and intervening build priorities even when the original commitment was reasonable. This task uses the larger of the selected-to-best ratio and a margin-aware score $1 - (\text{bestScore} - \text{selectedScore})$ so that small snapshot-plus-outcome differences do not make the benchmark’s own reference heuristic fail.

A.4.20 DECIDE-PURSUE-LARGEST-ARMY

$$\begin{aligned} \text{pursueScore} = & 0.40 \cdot \text{knightCardRatio} + 0.30 \cdot \text{opponentArmyGap} \\ & + 0.20 \cdot \text{devCardAffordability} + 0.10 \cdot \text{VPImpact}. \end{aligned}$$

The rubric stated at the start of Section A.4.20 remains appropriate, but it should be evaluated over a 4–6 turn future window rather than immediately. Largest Army is not about one development-card purchase or one Knight; it is about whether a multi-turn commitment improves the player’s race standing without unacceptable economic damage. A hybrid intent-plus-outcome model is appropriate because pure outcome scoring would overreact to deck order and dice luck, while pure immediate scoring would ignore whether the strategic plan materialized. Like Longest Road, this task is implemented as a pending long-horizon episode. At decision time it snapshots player knights played, strongest opponent army rival and their knights played, whether either player currently has Largest Army, player visible score, player hand size, and development cards bought so far in benchmark long-term stats.

At finalization it computes:

$$C_{\Delta} = \max(0, \text{devCardsBoughtSinceSnapshot}), \quad K_{\Delta} = \max(0, K_{\text{self}, \text{end}} - K_{\text{self}, \text{start}}),$$

$$G_{\text{start}} = K_{\text{self}, \text{start}} - K_{\text{target}, \text{start}}, \quad G_{\text{end}} = K_{\text{self}, \text{end}} - K_{\text{target}, \text{end}},$$

$$\text{knightCardRatio} = \begin{cases} 1, & K_{\Delta} > 0 \text{ and } C_{\Delta} = 0, \\ 0, & K_{\Delta} = 0, \\ \text{clamp}\left(\frac{K_{\Delta}}{C_{\Delta}}\right), & \text{otherwise,} \end{cases}$$

$$\text{opponentArmyGap} = \text{clamp}\left(\frac{(G_{\text{end}} - G_{\text{start}}) + 3}{6}\right),$$

$$\Delta R = \text{currentHandSize} - \text{snapshotHandSize},$$

$$\text{devCardAffordability} = \text{clamp}\left(0.6 + \frac{\Delta R}{10} - 0.08C_{\Delta}\right),$$

$$\text{vpImpact} = \text{clamp} \left(\frac{\Delta \text{visibleVP} + H_{\text{army, end}}}{3} \right).$$

Here $H_{\text{army, end}}$ is 1 if the player has Largest Army at finalization and 0 otherwise. The final alternatives are

$$\begin{aligned} \text{pursueScore} &= \text{clamp}(0.40 \cdot \text{knightCardRatio} + 0.30 \cdot \text{opponentArmyGap} \\ &\quad + 0.20 \cdot \text{devCardAffordability} + 0.10 \cdot \text{vpImpact}), \\ \text{deferScore} &= \text{clamp}(0.35 \cdot (1 - \text{opponentArmyGap}) \\ &\quad + 0.25 \cdot (1 - \text{knightCardRatio}) + 0.20 \cdot R_{\text{safe}} \\ &\quad + 0.20 \cdot (1 - H_{\text{army, end}})). \end{aligned}$$

Here R_{safe} is 1 if $\Delta R \geq 0$ and 0.4 otherwise.

The task is only instantiated when there is a real army-race signal: either player already holds Largest Army, the player already has at least four Knights played, the strongest rival already has at least five Knights played, or the player already has at least three Knights played and can immediately afford another development card.

The threshold for `decide-pursue-largest-army` is 0.30 because Largest Army pursuit is a long-horizon commitment and should be scored with delayed outcome evidence without over-penalizing reasonable defer-versus-pursue choices. This task uses the larger of the selected-to-best ratio and a margin-aware score $1 - (\text{bestScore} - \text{selectedScore})$ so that moderate snapshot-plus-outcome differences do not cause the benchmark’s own reference heuristic to fail.

B BENCHMARK USAGE AND EVALUATION DOCUMENTATION

This appendix documents how to use `CatanBench` outside the paper. The same runnable instructions are provided in `evaluation/README.md`; this appendix records the benchmark interface, required resources, evaluation scripts, and result interpretation in the paper itself. All code for the agent and benchmark is found in: <https://github.com/DhrvM/CatanAgent>

B.1 WHAT THE BENCHMARK EVALUATES

`CatanBench` records both full-game outcomes and task-level decisions. Full-game outcomes include win rate, final victory points, rounds to win, and average turn latency. Task-level outcomes include task score, task success/failure, task category, selected action, legal alternatives, and benchmark explanation. The benchmark covers spatial placement, road direction, robber decisions, trade generation/response, build prioritization, development-card timing, discard strategy, bank trades, and long-horizon Longest Road/Largest Army commitments.

The live task definitions, metric directions, and metric weights are implemented in `Environment/server/benchmarkDefinitions.js`. Task scoring logic is implemented in `Environment/server/benchmarkEvaluation.js`. Persistent benchmark storage, leaderboards, slices, and live logs are implemented in `Environment/server/benchmarkStore.js`.

B.2 REQUIRED RESOURCES

No external dataset is required. The benchmark is generated from live Catan games and server-side task evaluators. Required local resources are:

- `Environment/server/benchmarkDefinitions.js`: task descriptions, metric directions, and metric weights.
- `Environment/server/benchmarkEvaluation.js`: task scoring and benchmark payload evaluation.
- `Environment/server/benchmarkStore.js`: benchmark data storage, leaderboards, slice comparisons, and logs.
- `Agent/training/run_self_play.py`: automated heuristic self-play baseline runner.

-
- `evaluation/`: runnable evaluation scripts and result-export utilities.
 - `benchmark_paper/Result Images/`: validation evidence images used in this paper.

Generated resources are written locally:

- `Agent/training/results/*.json`: self-play baseline outputs.
- `Environment/server/benchmark-data/`: benchmark records written by the local server.
- `evaluation/results/<timestamp>/`: exported benchmark API snapshots.

OpenAI API access is optional. It is needed only for ReAct or multi-agent strategy evaluations that call GPT models. The heuristic baseline and server-side benchmark tests do not require an external API key.

B.3 DEPENDENCIES AND SETUP

The benchmark requires Python 3.10 or later, Node.js 18 or later, npm, Python packages from `requirements.txt`, and server packages from `Environment/server/package.json`. From the repository root:

```
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt

cd Environment/server
npm install
cd ../../
```

The browser client is optional for evaluation but useful for inspecting games and dashboards:

```
cd Environment/client
npm install
cd ../../
```

B.4 RUNNING THE BENCHMARK SERVER

Start the local server in a separate terminal:

```
cd Environment/server
npm start
```

The default local server is `http://localhost:3001`. To clear previous local benchmark records before a fresh run:

```
cd Environment/server
npm run benchmark:clear
```

B.5 VALIDATING BENCHMARK CODE

The server includes benchmark unit and observation tests. From the repository root:

```
./evaluation/run_benchmark_tests.sh
```

This runs benchmark observation tests, benchmark evaluation tests, resource heuristic tests, and longest-road cycle tests.

1080 **B.6 RUNNING THE HEURISTIC AGENT BASELINE**

1081

1082 The paper uses ten games of heuristic bot instances as the simple agent baseline. With the server

1083 running:

1084

```
1085 ./evaluation/run_heuristic_baseline.sh \
1086 --games 10 --players 3 --parallel 1
```

1087

1088 The output is written to:

1089

```
1090 Agent/training/results/heuristic_baseline_10_games.json
```

1091

1092 For a shorter debugging run:

1093

```
1094 ./evaluation/run_heuristic_baseline.sh \
1095 --games 1 --players 3 --timeout-s 600
```

1096

1097 **B.7 RUNNING LLM AGENT EVALUATIONS**

1098

1099 For ReAct or multi-agent strategy evaluations, start or join a benchmark-enabled game and launch

1100 agents against the same game code:

1101

```
1102 python -m Agent.main --mode react --server http://localhost:3001 \
1103 --game-code ABCDEF --name ReactAgent
```

1104

```
1105 python -m Agent.main --mode multi --server http://localhost:3001 \
1106 --game-code ABCDEF --name StrategyAgent
```

1107

1108 The benchmark-enabled server records task telemetry when benchmark mode is active for the game.

1109 The heuristic self-play runner is the most reproducible baseline path because it programmatically

1110 creates games, launches benchmark calibration agents, starts the game, waits for completion, computes rewards, and writes JSON summaries.

1111

1112 **B.8 EXPORTING RESULTS**

1113

1114 With the server running, export benchmark API state from the repository root:

1115

```
1116 ./evaluation/export_benchmark_results.py \
1117 --server http://localhost:3001
```

1118

1119 This creates a timestamped directory under `evaluation/results/` with:

1120

- 1121 • `definitions.json`: task definitions, categories, metric directions, and metric weights.
- 1122 • `overview.json`: aggregate benchmark overview and top entities.
- 1123 • `leaderboard_agents.json`: agent-level leaderboard.
- 1124 • `leaderboard_players.json`: player-level leaderboard.
- 1125 • `slices.json`: comparisons by benchmark, map, opponent policy, scenario tags, and
- 1126 task category.
- 1127 • `live_log.json`: recent task/action events with score and pass/fail metadata.
- 1128 • `manifest.json`: export metadata and endpoint mapping.

1129

1130

1131 To choose a specific output directory:

1132

```
1133 ./evaluation/export_benchmark_results.py \
--out evaluation/results/my_run
```

1134 **B.9 INTERPRETING RESULTS**
1135
1136 Start with `overview.json`. Its `totals.runs`, `totals.games`, `totals.tasks`, and
1137 `totals.telemetryRecords` fields show how much data has been collected. `topAgents`
1138 and `topPlayers` show the highest aggregate benchmark entries.
1139
1140 Then inspect `leaderboard_agents.json`. The `overallScore` field is the weighted score
1141 used for ranking. `rawMetrics` contains direct values such as win rate, average final victory points,
1142 task success rate, and latency. `normalizedMetrics` contains direction-normalized scores in
1143 which higher is better. `taskBreakdown` and `taskResults` identify which benchmark tasks
1144 passed or failed.
1145
1146 Use `slices.json` to check robustness across task category, map layout, opponent policy set,
1147 and scenario tags. A useful benchmark should expose strengths and weaknesses across these slices
1148 rather than only producing one undifferentiated score. Use `live_log.json` for auditability: each
1149 entry links an observed action to its benchmark task, score, pass/fail result, and explanation.
1150
1151 Evaluation reports should include overall score, gameplay/reasoning/operations scores, win rate,
1152 average final victory points, average rounds to win, task success rate, average task score, per-task
1153 breakdowns, average latency per turn, number of games, number of task events, agent version, and
1154 any map/opponent/scenario filters used.
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187